



前端签名防御策略

协议韧性、AI 攻击者与签名校验

HackProve World 2026 白帽世界大会中国·澳门

主题 06：前端签名防御策略

本讲义基于主题 06 的 transcript、相对/绝对时间轴字幕、截图候选约定，以及 official_references.md 中允许使用的官方资料。时间边界为 01:42:11–02:06:20。

目录

摘要	2
1 术语表：先把边界说清楚	2
2 从一次搜索请求看签名协议	5
3 AI 攻击者改变了什么	6
4 前端签名的四类真实风险	6
4.1 重放：旧票据重新入场	6
4.2 参数污染：签名字段与业务字段错位	6
4.3 时钟偏差：过严会伤用户，过宽会养攻击	7
4.4 前端密钥暴露：别把玻璃柜当保险箱	7
5 四层防御：提高协议韧性	7
5.1 服务端重算：让信任回到服务端	7
5.2 短时效：给请求设置保质期	7
5.3 绑定会话：把签名贴回具体人和场景	8
5.4 异常检测：承认前端会被拆，靠观测收口	8
6 如何评估：ASR 与 Recall	8
7 落地建议：把签名当成防线之一	9
8 总结	9
9 官方参考	10



图 1: 主题 06 截图: Signature Protocol 页面给出客户端参数、签名函数与服务端验证的基本结构，是解释 nonce、timestamp、secret 与 signature 的入口图。²

摘要

Albert 的分享把前端签名放进一个更现实的攻防场景：攻击者已经有 AI agent、工具、记忆、阶段化计划和反复试错能力，防守方仍要让协议在这样的压力下保持可验证、可观测、可收敛。报告开头用手机搜索请求举例：客户端把设备信息、请求时间、关键词等字段发给服务端，服务端需要判断这是一名真实用户在使用应用，还是脚本或攻击者在伪造请求。签名协议的核心价值就在这里：它给消息加上一层可校验的“封蜡”。

封蜡这个类比很重要。古代信件上的蜡封不能阻止信被抢走，却能让收信人发现信是否被拆过、换过、晚到得离谱。前端签名也一样：它无法让运行在用户设备上的密钥永远隐身，却能通过服务端重算、短时效、绑定会话、异常检测和持续红队评估，提高攻击者的成本，降低批量伪造与重放的成功率。

1 术语表：先把边界说清楚



图 2: 主题 06 截图: Transition 页面展示 planner、worker 与 attacker 在协议测试中的状态转换, 适合说明 AI attacker 如何观察、变异并验证攻击假设。⁴



图 3: 主题 06 截图: Attack-Defense 页面把开发者防护与攻击者变异策略并置, 突出签名协议需要同时抵抗参数篡改、过期请求和重放请求。⁶

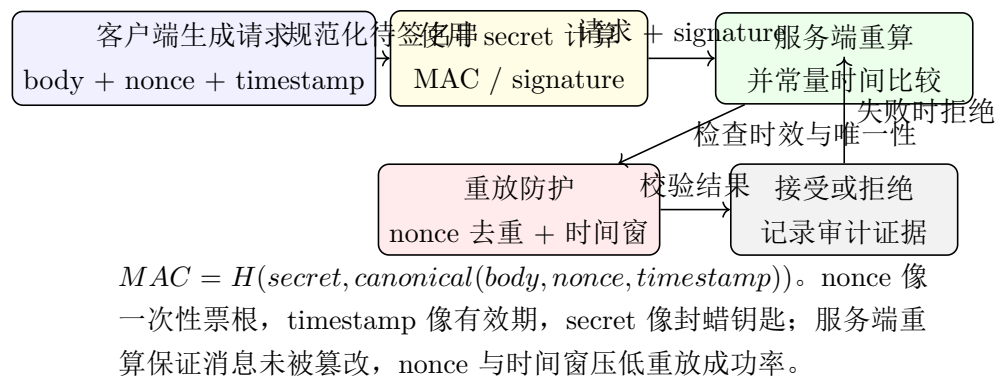


图 4: 主题 06 机制图：签名生成、服务端校验与重放防护的闭环；关键控制点是规范化、secret 保护、服务端重算、nonce 去重与 timestamp 时间窗。⁸

术语	定义与直观解释
签名协议	一组约定好的消息构造、摘要计算、密钥使用、时间窗口和服务端校验规则。它的目标是让服务端判断“这条请求是否由预期客户端按预期规则生成，字段是否被改动，是否仍在有效时间内”。形式上可写为 $s = \text{Sign}_k(m)$ ，其中 m 是待保护消息， k 是密钥或私钥， s 是签名或消息认证码。
nonce	一次性随机数或唯一值，常放在请求中防止同一条消息被反复使用。nonce 像票据上的唯一流水号；同一流水号如果第二次出现，服务端就应该警觉。
timestamp	时间戳，用来表示请求生成时间。它让请求拥有“保质期”，例如只允许在 ± 300 秒窗口内通过。时间戳像食品日期，不能证明食品美味，却能帮助拒绝明显过期的包裹。
MAC/signature	MAC 是 Message Authentication Code，常见实现是 HMAC；signature 通常指数字签名。二者都把消息和某种秘密材料绑定起来。HMAC 偏向“共享密钥双方互相校验”，数字签名偏向“私钥签名、公钥验证”。
重放攻击	攻击者不理解或不破解签名算法，只复制一条曾经合法的请求，在之后再次发送。它像拿着别人用过的入场券重新排队。nonce、timestamp 和服务端已用记录是常见抑制手段。
协议韧性	协议在被观察、变异、批量尝试、局部泄露和自动化攻击时，仍能维持核心安全性质的能力。韧性不等同于永不失守，它强调退化可控、证据可见、修复有抓手。

术语

定义与直观解释

AI 攻击者

本讲沿用前序主题对 Agent 的定义，把 AI 攻击者理解为可围绕目标进行多步计划、调用工具、保存阶段结果并不断试错的自动化攻击系统。它把观察、假设、变异、验证连成循环。

2 从一次搜索请求看签名协议

报告开头的例子很朴素：用户在手机里搜索“123”，客户端向服务端发出 HTTP 请求，请求头或请求体里可能带有设备标识、请求时间、关键词等字段。服务端返回搜索结果。问题随即出现：服务端怎样区分正常应用发出的请求和攻击者伪造的数据抓取请求？

签名像消息封蜡

客户端先把关键字段按固定规则整理成待签名消息，再用密钥材料生成 MAC 或 signature。服务端收到请求后，用同一套规则重算一次。如果重算值与请求携带的签名一致，且 nonce、timestamp、会话状态也满足要求，请求才进入后续业务处理。蜡封提供的是“可验性”，无法充当魔法护盾。

一个典型的 HMAC 形式可以写成：

$$\text{mac} = \text{HMAC}_k(\text{method} \parallel \text{path} \parallel \text{bodyHash} \parallel \text{timestamp} \parallel \text{nonce} \parallel \text{sessionId})$$

其中 $\parallel$ 表示字段拼接。实际工程中还要定义字段排序、编码方式、大小写处理、空值处理、JSON 规范化和换行规则。签名协议最怕“双方都觉得自己懂规则”，因为一点点规范化差异就可能造成误拒，或者给参数污染留下缝隙。

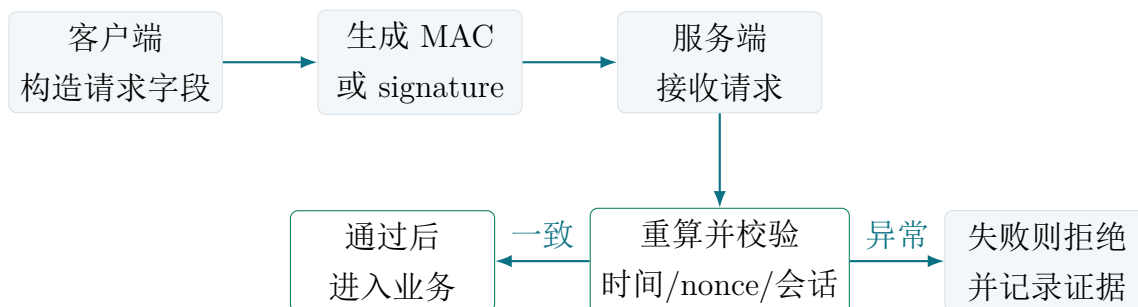


图 5: 签名生成、服务端重算与异常拒绝的基本链路。

3 AI 攻击者改变了什么

本讲的重点并非某个单点漏洞，而是 secure protocol resilience against attack。讲者强调，他更愿意谈 agent design，因为真实结果涉及保密。这个取舍本身说明了问题：当攻击者拥有自动化 agent，前端签名防御要面对的就不再是一次手工抓包，而是持续的观察、假设、变异和验证。

AI 攻击者通常会做四件事：

1. **观察**：收集请求样本、字段变化、错误响应和时间窗口。
2. **假设**：猜测哪些字段参与签名，哪些字段只作业务参数，哪些字段用于设备或会话绑定。
3. **变异**：改变关键词、时间、设备状态、编码形式、字段顺序和固定模式。
4. **验证**：根据响应差异判断假设是否成立，再把经验写入记忆或下一阶段计划。

官方资料中，OpenAI 的 *New tools for building agents* 把 agent 描述为代用户完成任务的系统，涉及工具、编排、观测与 tracing；OpenAI 的 *Advancing red teaming with people and AI* 也强调自动化方法可用于规模化生成测试样例。放到本讲语境里，AI 攻击者就是把“猜签名规则”这件事变成可迭代的实验流水线。

4 前端签名的四类真实风险

4.1 重放：旧票据重新入场

重放攻击的危险在于朴素。攻击者可以完全不理解 MAC 的内部计算，只要抓到一条合法请求，就尝试再次发送。没有 timestamp 时，请求像没有日期的入场券；没有 nonce 和已用记录时，同一张票可以反复排队。

防御的基本式子可以写成：

$$\text{valid}(r) = [|\text{now} - t_r| \leq \Delta] \wedge [n_r \notin N_{\text{used}}] \wedge [\text{mac}_r = \text{recompute}(r)]$$

其中 t_r 是请求时间， Δ 是允许时间窗口， n_r 是 nonce， N_{used} 是服务端记录的已使用 nonce 集合。

4.2 参数污染：签名字段与业务字段错位

参数污染常见于字段规范不清。客户端签了一个字段集合，服务端业务却读取另一个字段集合；或者服务端校验时使用的 JSON 规范化方式与业务解析方式不一致。攻击者会

寻找这种“签名看到 A，业务执行 B”的错位。签名协议可以用“输入源到敏感动作”的视角审查：输入字段是来源，鉴权、支付、查询、导出等动作是敏感汇点，中间每一次解析和转换都要可追踪。

4.3 时钟偏差：过严会伤用户，过宽会养攻击

timestamp 的窗口太短，移动网络、弱网、设备时钟漂移会造成误拒；窗口太长，重放攻击的可用时间变宽。这里没有免费的午餐。工程上需要按业务风险分层：普通搜索请求可以容忍较宽窗口，高价值动作应使用更短窗口、更强会话绑定和二次确认。

4.4 前端密钥暴露：别把玻璃柜当保险箱

问答环节里，现场问题直指痛点：Web JS、App Java 层或 Native 层里的算法、函数和 secret key，攻击者总有机会逆向、Hook 或动态观察。讲者的回答很直接：这永远是攻击成本与防御成本的问题。把代码移到 WebAssembly、Native 层或 VM 可以提高可读性门槛；动态解密、内存地址间接读取和环境检测也能增加成本；但只要秘密材料必须在用户设备上参与计算，防守方就应假设它有泄露概率。

前端密钥的硬边界

前端签名不能被设计成“只要有签名就等于可信用户”。更稳妥的判断是：签名证明“请求符合某个客户端侧生成流程”，还需要服务端身份、会话、设备、行为、频率和风险模型共同判定。把所有信任压在前端密钥上，等于把银行金库钥匙长期放在大厅展柜里。

5 四层防御：提高协议韧性

5.1 服务端重算：让信任回到服务端

服务端不应接受客户端给出的“我已经算过了”。正确做法是服务端拿到请求后按协议重算 MAC/signature，并且用常量时间比较降低侧信道风险。更关键的是，服务端只使用自己掌握的可信状态：服务端保存的会话密钥、设备绑定状态、nonce 使用记录和风险评分。

5.2 短时效：给请求设置保质期

短时效降低重放窗口。对低风险请求，时间窗口可以稍宽，减少误拒；对高风险请求，窗口应短，并辅以 nonce、一次性挑战或服务端下发的临时令牌。临时令牌像临时门禁卡，

即使被拍照，也很快失效。

5.3 绑定会话：把签名贴回具体人和场景

签名消息里应绑定会话、设备、用户状态和业务动作。绑定的价值在于缩小搬运空间：A 会话里的签名不能直接搬到 B 会话，搜索请求里的签名不能直接套到导出数据或修改资料。可以把绑定理解为给封蜡写上收件人、房间号和日期，减少“拿别人的封蜡盖自己的信”的机会。

5.4 异常检测：承认前端会被拆，靠观测收口

异常检测关注行为，而非只看单包是否签名正确。可观测信号包括签名失败率、nonce 重复率、时间戳偏移分布、同设备多账号、同账号多设备、请求频率、字段熵值、环境检测异常和业务结果异常。在前端签名场景里，应把自动化访问限制在受控边界内，并在越界或异常聚集时触发更强校验。

协议韧性检查清单

- 签名字段是否覆盖 method、path、body hash、timestamp、nonce、sessionId 与高风险业务参数。
- 服务端是否拥有唯一可信重算路径，是否拒绝客户端自报的中间结果。
- nonce 是否有足够随机性或唯一性，服务端是否记录已用 nonce。
- 时间窗口是否按业务风险分层，是否考虑弱网与设备时钟偏差。
- 前端密钥泄露后，服务端是否能够通过轮换、降级、风控和封禁收敛损失。
- 异常检测是否覆盖签名失败、重放、参数变异、环境异常和自动化高频访问。

6 如何评估：ASR 与 Recall

本主题可以借用后续主题会完整定义的 ASR 与 Recall。这里给出必要的工作定义，用于评估签名协议韧性：

$$ASR = \frac{\text{成功通过校验的攻击请求数}}{\text{攻击尝试总数}}$$

ASR 越低，表示攻击者通过签名校验与风控的比例越低。它适合衡量“攻击能不能打穿”。

$$Recall = \frac{TP}{TP + FN}$$

Recall 是召回率， TP 表示被正确识别的异常请求， FN 表示漏掉的异常请求。它适合衡量“检测系统有没有漏掉”。签名防御的现实目标是同时压低 ASR、提高 Recall，并控制误伤率。若 Recall 高但误伤正常用户，业务会被防御拖垮；若用户体验很好但 ASR 高，协议就只是装饰。

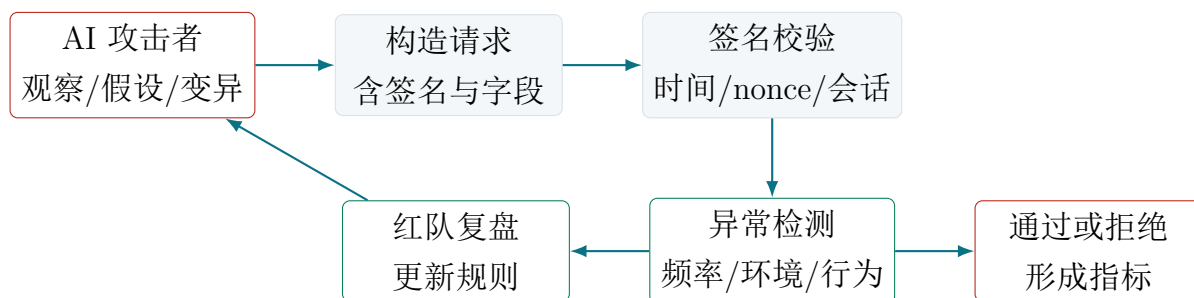


图 6: 协议韧性评估循环：攻击者迭代，防守方也必须把证据回流。

7 落地建议：把签名当成防线之一

前端签名最容易被误用成“万能鉴权”。真实工程里，它更适合承担三类职责：抬高批量伪造成本、发现协议被重放或篡改、为风控系统提供结构化证据。它不适合单独承担身份真实性、设备可信性和业务授权的全部责任。

一套更稳的落地顺序如下：

1. **先收口协议规范**：明确字段、排序、编码、body hash、错误码和日志字段，避免客户端与服务端各算各的。
2. **再做服务端状态**：会话密钥、nonce 记录、时间窗口、密钥轮换和风险等级应在服务端统一管理。
3. **然后做分层校验**：普通请求使用基础签名，高价值动作叠加短时效挑战、二次确认或更严格风控。
4. **最后做红队闭环**：用 AI attacker 思路模拟观察、变异、重放和批量尝试，把失败与成功样本沉淀成回归集。

8 总结

本讲的关键结论很硬：前端签名不能彻底解决前端密钥可被逆向的问题。它的价值在于把请求变成可验证对象，把伪造、篡改和重放变成有成本、有痕迹、有窗口的行为。签

名像消息封蜡，服务端重算像验蜡，短时效像保质期，绑定会话像收件人地址，异常检测像门口的摄像头和巡检日志。

AI 攻击者让这个问题更尖锐。它会长时间观察，会自动变异，会记住失败，会把绕过经验转移到下一轮。协议韧性要求防守方也具备同样的循环：规则清晰、状态可信、指标可量化、证据能回流、修复能迭代。把签名放在这套循环里，它是有用的；把签名当成孤立护身符，它会很快被现实拆穿。

9 官方参考

- OpenAI. *New tools for building agents*. <https://openai.com/index/new-tools-for-building-agents/>
- OpenAI. *Advancing red teaming with people and AI*. <https://openai.com/index/advancing-red-teaming-with-people-and-ai/>

资料边界：以上条目均来自素材根目录的 `official_references.md`。正文没有使用博客、媒体、论文解读站或社区帖子作为外部依据。